Project Heimdall – Engineering Specification (for Etel / Vibe Coder)

Scope: implementation of the perception, geolocation, fusion, explainability and evaluation layers. Data ingestion / dataset preparation is explicitly out of scope for this document. Generated on 2026-01-30.

Global goal

Build a clean, testable and reproducible system that (1) detects objects in images, (2) proposes multiple geolocation candidates, (3) verifies and fuses them using probabilistic models to produce a posterior distribution and uncertainty, and (4) exposes explainable results through a lightweight UI and reports.

Module A – Detection interface

- Implement a detector abstraction layer (interface) supporting YOLO■OBB as the first backend.
- Inputs: image or video frame tensor.
- Outputs: list of detections with: class, confidence, oriented bounding box (4 points), heading (deg).
- Expose optional embedding vector per detection for later tracking.
- All detectors must conform to a single Python interface: detect(image) -> List[Detection].
- Provide a lightweight benchmark script for detector-only evaluation.

Deliverables

- core/detection/base.py – abstract detector interface.
- core/detection/yolo_obb.py – YOLO implementation.
- schemas/detection.py – pydantic model.
- tools/benchmark_detector.py – detector-only timing + counts.

Module B – Geolocation candidate generation

- Implement a geolocation candidate provider interface.
- Backend initially uses an embedding retrieval model (GeoCLIP / GeoFT style).
- Given an image, return top■N candidate locations.
- Each candidate must contain latitude, longitude, retrieval score, and optional matched tile or landmark id.
- Do not return a single best location. Always return a ranked list of candidates.

Deliverables

- core/geo/candidate_provider.py – interface.
- core/geo/geoclip_provider.py – first implementation.
- schemas/geo_candidate.py – pydantic schema.
- tools/run_geo_candidates.py – offline test runner.

Module C – Probabilistic verification and fusion (core differentiator)

This module replaces heuristic scores by a probabilistic posterior over candidate locations.

- Inputs: detections, geo candidates, timestamp (if available), and verification signals.
- Verification signals (initial scope):
- - Shadow consistency: angular residual between observed shadow direction and theoretical sun azimuth at candidate location and time.
- - Terrain consistency: stub residual (future improvement, keep interface ready).
- Define likelihood models:
- - P(shadow_residual | location) using Gaussian or von■Mises distribution.
- - P(terrain_residual | location) using Gaussian (can be disabled by config).
- Convert retrieval score into probability using temperature scaling.
- Fuse all signals in log■space and normalize to obtain posterior weights over candidates.
- Output top■K candidates with probabilities.

Uncertainty estimation

- Convert lat/lon to projected metric coordinates.
- Compute weighted mean and covariance using posterior weights.
- Extract uncertainty ellipse (major axis, minor axis, orientation).
- Provide uncertainty_radius_m for UI and reporting.

Deliverables

- core/logic/likelihoods.py – likelihood models.
- core/logic/fusion.py – fusion engine.
- schemas/fusion.py – output schema.
- tools/calibrate_fusion.py – temperature scaling and calibration utility.

Module D – Explainability layer

- For each candidate location, store an evidence object.
- Evidence must include: retrieval score, shadow residual, terrain residual, likelihood values and final posterior weight.
- Generate a compact explanation text template per candidate.
- Expose explainability fields in JSON outputs and UI.

Deliverables

- schemas/evidence.py – evidence model.
- Extend fusion output with evidence.
- tools/generate_ui_data.py – include explainability fields.

Module E – Temporal / multi■view consistency (tracking)

- Implement a track abstraction grouping detections across frames or posts.
- Association strategy: embedding similarity + spatial plausibility + temporal proximity.
- For each new observation in a track, update the posterior distribution using the previous posterior as a prior.
- Add soft motion constraints (maximum speed, heading consistency).
- Tracking must be optional and switchable by configuration.

Deliverables

- core/logic/tracking.py – association logic.
- core/logic/filtering.py – Bayesian update / filter.
- schemas/track.py – track schema.
- tools/run_sequence_eval.py – small evaluation harness.

Module F – UI and reporting

- Map view: candidate points, posterior-weighted final estimate, uncertainty ellipse.
- Table view: candidates with residuals, likelihoods and weights.
- Object view: detection crops and classes.
- Track view: timeline and location refinement over time.
- Audit export: one JSON file per image with all intermediate signals.

Deliverables

- dashboard/ extensions: candidate table, uncertainty overlay, track timeline.
- tools/run_tests_report.py – generate HTML report.
- tools/export_audit.py – per-image audit JSON.

Module G – Evaluation and reproducibility

- One command evaluation producing a versioned JSONL file and HTML report.
- Metrics: top■1 / top■5 accuracy (when ground truth exists), calibration error (ECE), average uncertainty radius.
- Ablation runs: retrieval only vs fusion vs fusion + temporal.
- All runs must store configuration snapshot and git commit hash.

Deliverables

- tools/run_all.py – full pipeline runner.
- tools/eval_metrics.py – metrics and calibration.
- docs/REPRODUCIBILITY.md – exact steps.

Non■goals (explicit)

- No data ingestion connectors, scraping logic or dataset construction in this phase.
- No LLM or text processing components.
- No operational or real■world deployment claims.
- No automated decision or targeting functionality.

End of specification.